

```
In [1]: %matplotlib widget
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
import numpy as np
import scipy.constants as c
from scipy.optimize import curve_fit
from scipy.stats import chi2
from pathlib import Path
from adjustText import adjust_text
from matplotlib.widgets import Button

def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))
```

```
In [2]: base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/"234")      # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)
```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Messwerte\234

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Auswertungen\234\Code\Diagramme

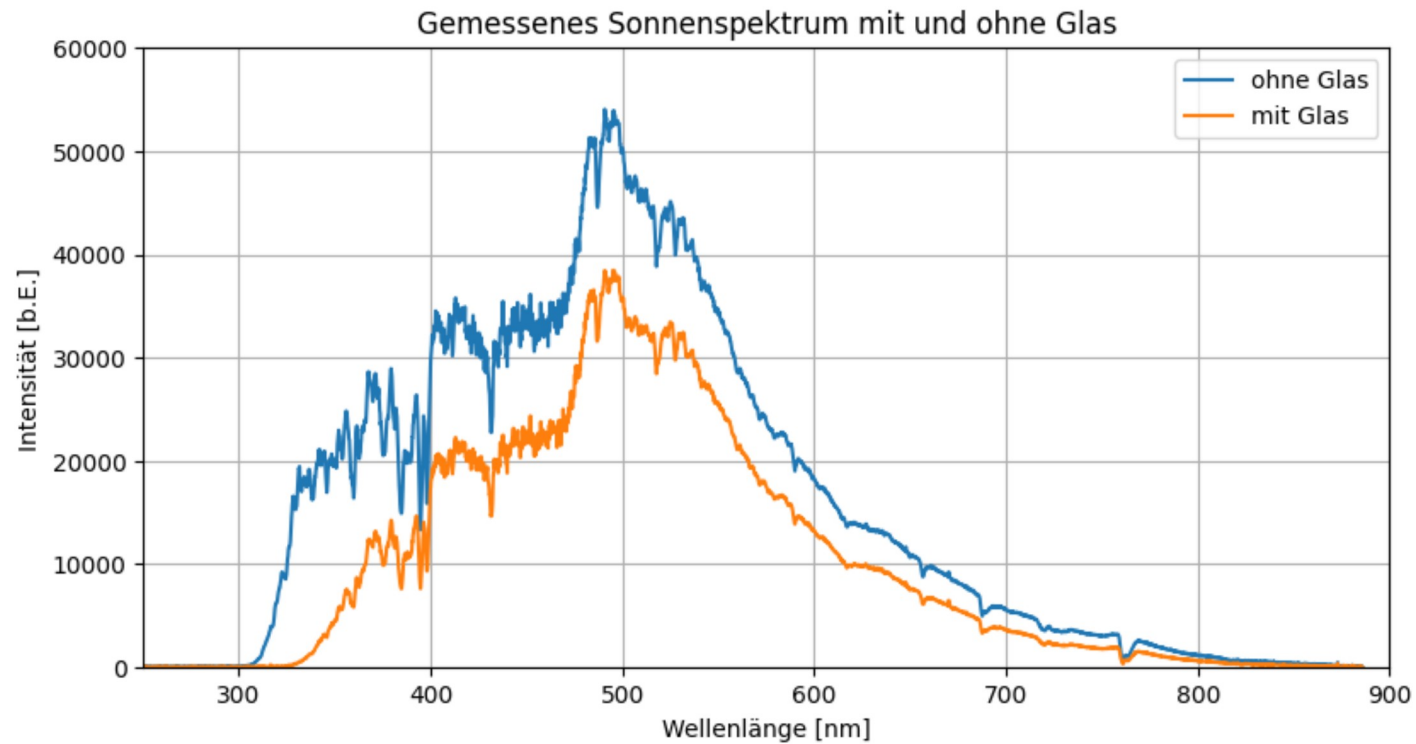
Nr.1 Sonnenspektrum mit und ohne Glasfenster

```
In [ ]: lambda_ohneGlas, intensity_ohneGlas = np.loadtxt(Messwerte_path/"Nr.1"/"blauerhimmel_ohneglas.txt", skiprows=17,
converters={0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)

lambda_mitGlas, intensity_mitGlas = np.loadtxt(Messwerte_path/"Nr.1"/"blauerhimmel_hinterglas.txt", skiprows=17,
converters={0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)

plt.figure(figsize=figsize_cm(12,6))      # Standard Plot erstellen, der gespeichert wird.
plt.plot(lambda_ohneGlas, intensity_ohneGlas, label='ohne Glas')
plt.plot(lambda_mitGlas, intensity_mitGlas, label='mit Glas')
plt.title('Gemessenes Sonnenspektrum mit und ohne Glas')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.legend()
plt.grid()
plt.ylim((0,60000))
plt.xlim((250,900))
# plt.savefig(Ausgabe_path/"Nr.1_Himmel_m_o_G.png", format="png")
```

Out[]: (250.0, 900.0)



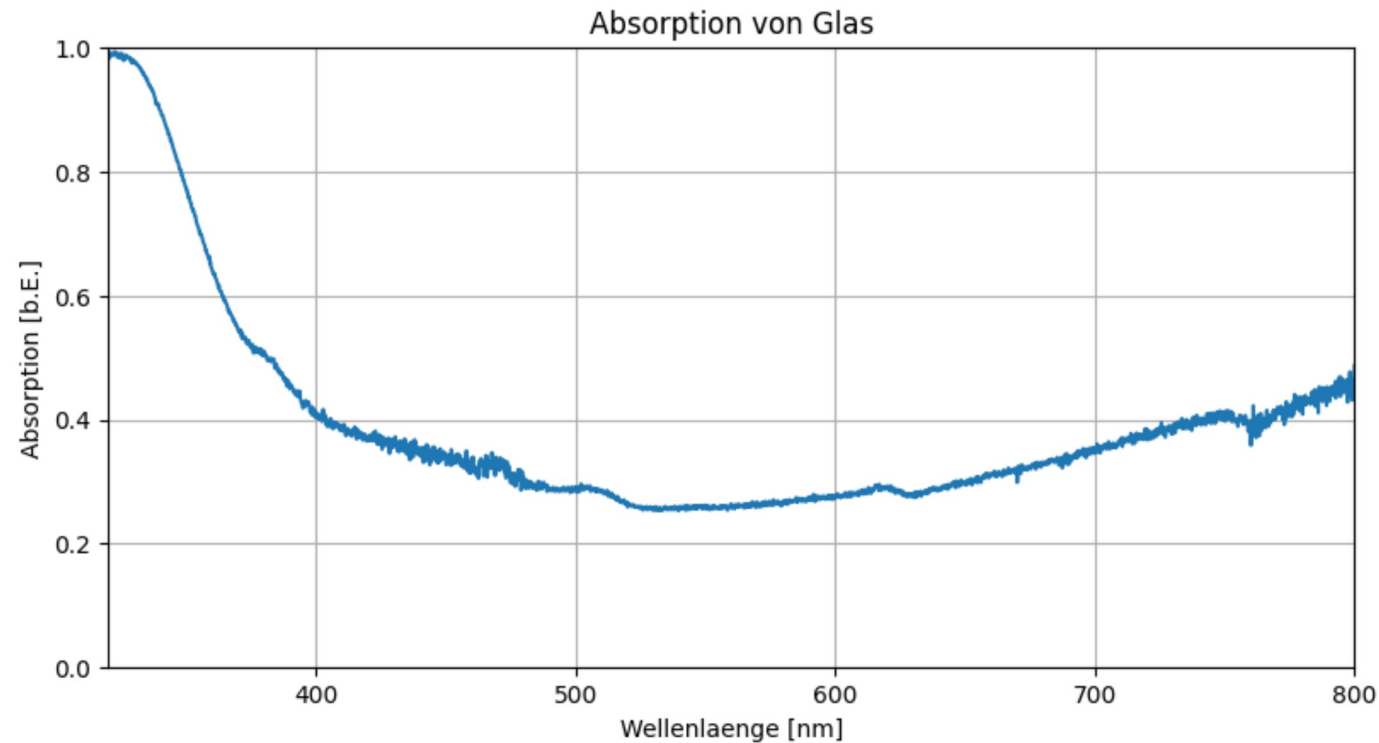
Nr.1.1 Absorption

In [4]: $A = 1 - \text{intensity_mitGlas} / \text{intensity_ohneGlas}$

```
plt.figure(figsize=figsize_cm(12,6))
plt.plot(lambda_ohneGlas,A)
plt.title('Absorption von Glas')
plt.xlabel('Wellenlaenge [nm]')
plt.ylabel('Absorption [b.E.]')
plt.grid()
plt.ylim((0,1))
plt.xlim((320,800))
# plt.savefig(Ausgabe_path/"Nr.1_Absorption_Glas.png", format="png")
```

Out[4]: (320.0, 800.0)

Figure



Nr.1.2 Hier wird es das erste Mal interessant: Sonnenspektrum mit Absorptionslinien und Balmer-Serie

```
In [ ]: lambda_sonneohneglas, intensity_sonneohneglas = np.loadtxt(Messwerte_path/"Nr.1"/"direktesonne_ohne_glas.txt", skiprows=17,
converters={0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)

bottom, top = plt.ylim()
plt.figure(figsize=figsize_cm(12, 6))
plt.plot(lambda_sonneohneglas, intensity_sonneohneglas,)
plt.vlines(x=[394, 397, 412, 431, 486.8, 517, 527, 589, 616.5, 656, 687, 760.5], ymin=0, ymax=max(bottom, top), colors='black', linewidths=0.7, linestyle='dotted', label='geschä
plt.title('Sonnenspektrum mit abgelesenen Absorptionslinien')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.ylim((0, 60000))
plt.xlim((350, 800))
balmerserie = {
    r'$H\_alpha$': 656.1,
    r'$H\_beta$': 486.1,
    r'$H\_gamma$': 434,
    r'$H\_delta$': 410.1
}
# in dieser Liste werden die Namen der einzuzeichnenden Linien gespeichert

plt.vlines(x=[410.1, 434, 486.1, 656.1], ymin=0, ymax=40000, colors='red', linewidths=0.7, label='Balmerserie')
for name, wl in balmerserie.items():
    # mit dieser Schleife werden die Linienlabels gesetzt
```

```

if not wl==410.1:
    plt.text(wl+2,15000,f"{name}\n({wl} nm)",color='red')
else:
    plt.text(wl+2,24000,f"{name}\n({wl} nm)",color='red')
texts=[]
geschätzte_absorptionslinien=[394.6,398,411.4,431.5,435.2,486.8,517.7,527.5,586.4,589.9,589.9,656.6,687.5,760.8] # mit der Maus erfasste Linien werden gespeichert
for i in geschätzte_absorptionslinien: # und gezeichnet
    t = plt.text(i, 50000, str(i), color="black", rotation=0,
        verticalalignment="bottom", horizontalalignment="left")
    texts.append(t)
adjust_text(texts) # soll Labels automatisch uncolliding machen, klappt manchmal
plt.axvline(x=586.5,color='goldenrod',label='Helium (Lit.  $\lambda=487.6\text{nm}$ ', linewidth=1)
plt.text(560,40000,str(486.5), color='goldenrod')
# plt.grid()
plt.legend()
# plt.savefig(Ausgabe_path/"Nr.1_FraunhoferLinien.png", format="png")

```

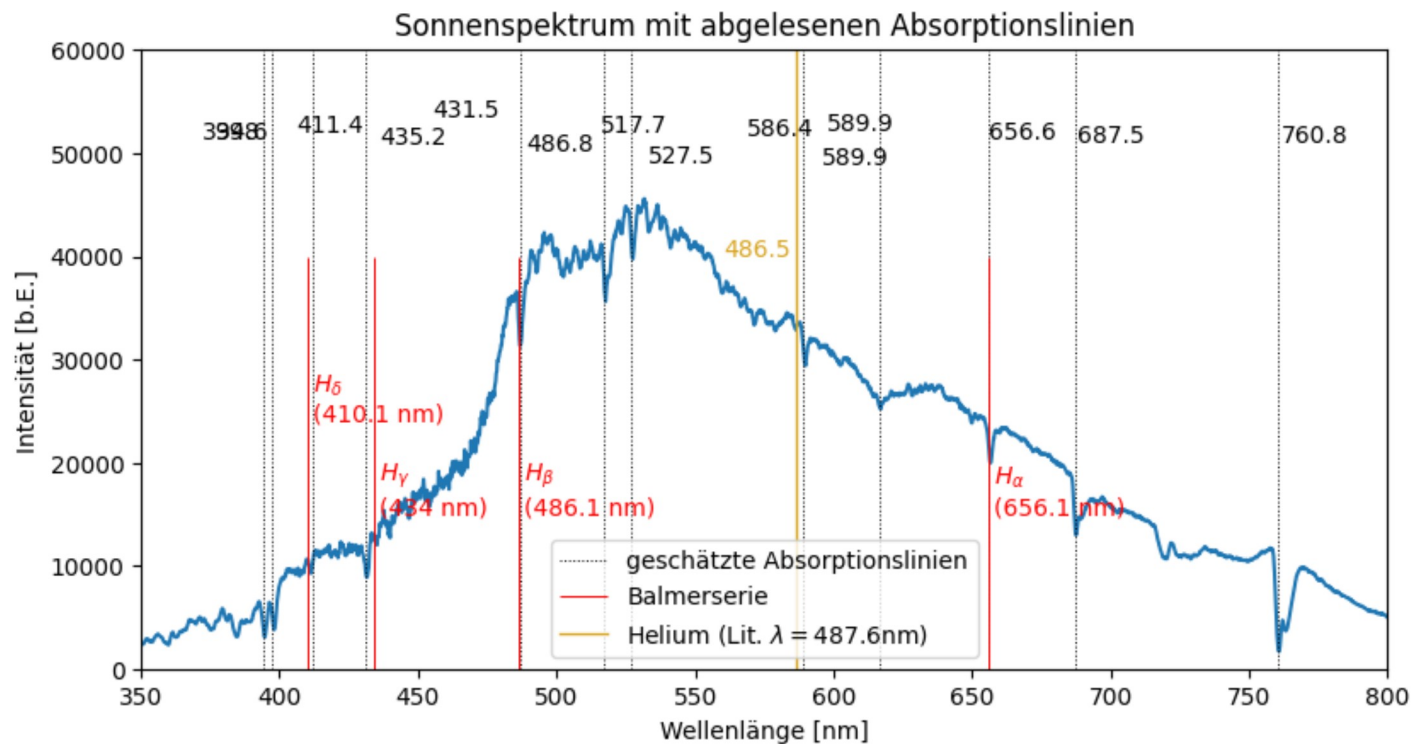
```

9 [ 0.27475631 -0.89015311]
10 [-0.47846447 -0.01190192]

```

Out[]: <matplotlib.legend.Legend at 0x1c4c450c2f0>

Figure



Nr.1.2 Sonnenspektrum mit allen Fraunhoferlinien

In [10]: `plt.figure(figsize=figsize_cm(12,6))`


```

plt.plot(lambda_sonneohneglas,intensity_sonneohneglas)
plt.title('Sonnenspektrum mit theoretischen Fraunhoferlinien')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.xlim((350,800))
plt.ylim((0,60000))
fraunhofer_lines = {
    "Sauerstoff": ("gray", [759.4, 686.7]),
    "Wasserstoff": ("red", [656.1, 486.1,434.1,410.1]),
    "Natrium": ("orange", [589.6, 589.0]),
    "Helium": ("purple", [587.6]),
    "Eisen & Calcium": ("brown", [527.0, 430.8]),
    "Magnesium": ("green", [518.4]),
    "Calcium": ("blue", [396.8, 393.4]),
}
handles=[]
for element, (color, wavelengths) in fraunhofer_lines.items():
    for wavelength in wavelengths:
        if not element=='Wasserstoff':
            plt.axvline(wavelength, color=color, linestyle="dotted",linewidth = 1)
        else:
            plt.axvline(wavelength, color=color, linewidth = 0.5)
for element, (color, _) in fraunhofer_lines.items():
    if not element=='Wasserstoff':
        handles.append(Line2D([0], [0], color=color, linestyle="dotted", label=element))
    else:
        handles.append(Line2D([0], [0], color=color, label=element))

texts=[]
plt.vlines(x=geschätzte_absorptionslinien,ymin=0,ymax=max(bottom,top), colors='black', linewidths = 0.5, label='geschätzte Absorptionslinien')
for i in geschätzte_absorptionslinien:
    t = plt.text(i, 50000, str(i), color="black", rotation=0,
        verticalalignment="bottom", horizontalalignment="left")
    texts.append(t)
adjust_text(texts)
handles.append(Line2D([0], [0], color='black', label='gemessene Linien'))
plt.legend(handles=handles,loc='center right')
# plt.savefig(Ausgabe_path/"Nr.1_FraunhoferLinientheoretisch.png", format="png")

```

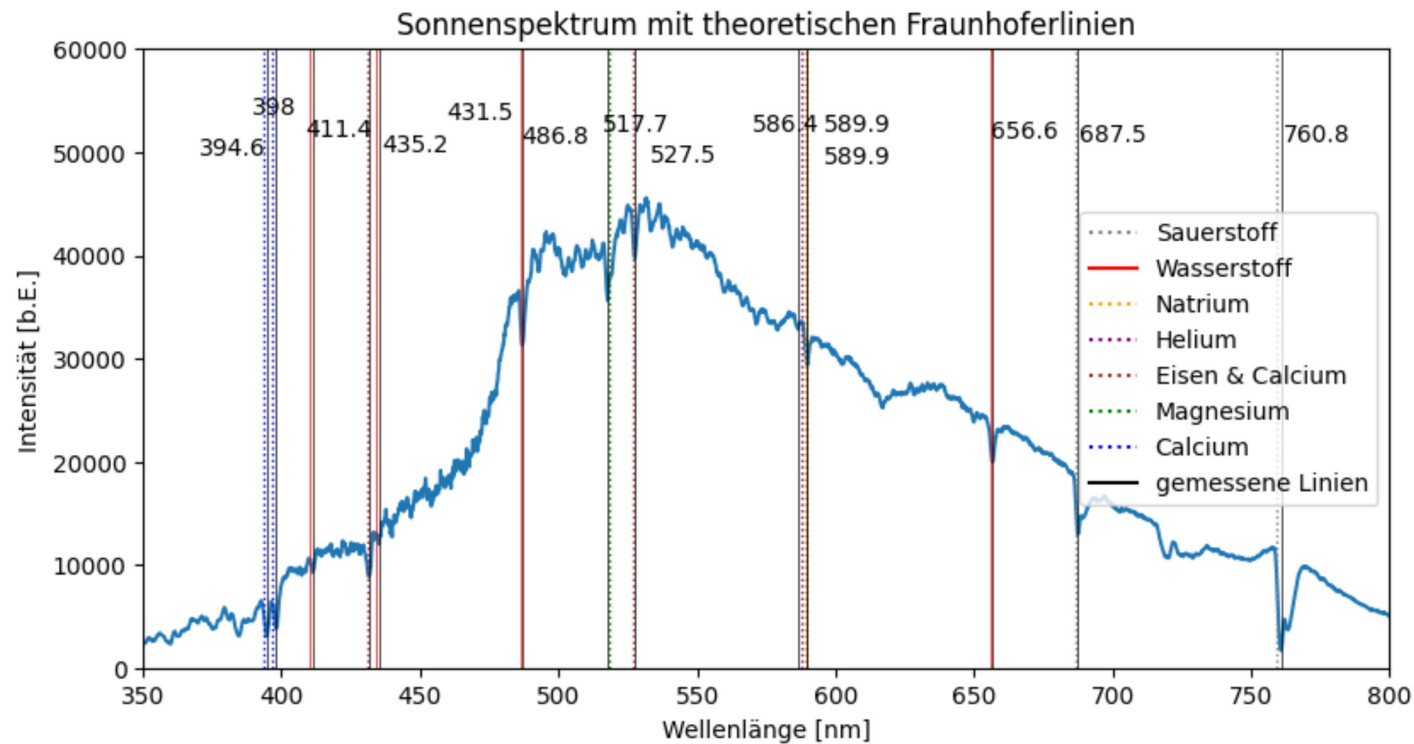
```

9 [-0.12713059  0.1726091 ]
10 [-0.17357367 -0.64139173]

```

Out[10]: <matplotlib.legend.Legend at 0x1c4c465fe00>

Figure



Nr.1.2 Ausgeben der LaTeX-Tabelle für gemessene und theoretische Fraunhoferlinien im Sonnenspektrum, berechnen der Sigma-Abweichung

```
In [56]: theoretische_Linien=[]
for element, (color, wavelengths) in fraunhofer_lines.items():
    for wavelength in wavelengths:
        theoretische_Linien.append(wavelength)
Theoretische_Linien=np.array(theoretische_Linien)
Theoretische_Linien.sort()
Geschätzte_absorptionslinien=np.array(geschätzte_absorptionslinien)
print(Theoretische_Linien)
print(Geschätzte_absorptionslinien)
sigma=np.round(abs(Theoretische_Linien-Geschätzte_absorptionslinien),1)
print(sigma)

for i,j,k in zip(Theoretische_Linien, Geschätzte_absorptionslinien, sigma):
    print(f"\num{{{i}}}&\num{{{j}}}&\num{{{k}}}\\"")
```

```
[393.4 396.8 410.1 430.8 434.1 486.1 518.4 527. 587.6 589. 589.6 656.1
686.7 759.4]
[394.6 398. 411.4 431.5 435.2 486.8 517.7 527.5 586.4 589.9 589.9 656.6
687.5 760.8]
[1.2 1.2 1.3 0.7 1.1 0.7 0.7 0.5 1.2 0.9 0.3 0.5 0.8 1.4]
\num{393.4}&\num{394.6}&\num{1.2}\\
\num{396.8}&\num{398.0}&\num{1.2}\\
\num{410.1}&\num{411.4}&\num{1.3}\\
\num{430.8}&\num{431.5}&\num{0.7}\\
\num{434.1}&\num{435.2}&\num{1.1}\\
\num{486.1}&\num{486.8}&\num{0.7}\\
\num{518.4}&\num{517.7}&\num{0.7}\\
\num{527.0}&\num{527.5}&\num{0.5}\\
\num{587.6}&\num{586.4}&\num{1.2}\\
\num{589.0}&\num{589.9}&\num{0.9}\\
\num{589.6}&\num{589.9}&\num{0.3}\\
\num{656.1}&\num{656.6}&\num{0.5}\\
\num{686.7}&\num{687.5}&\num{0.8}\\
\num{759.4}&\num{760.8}&\num{1.4}\\
```

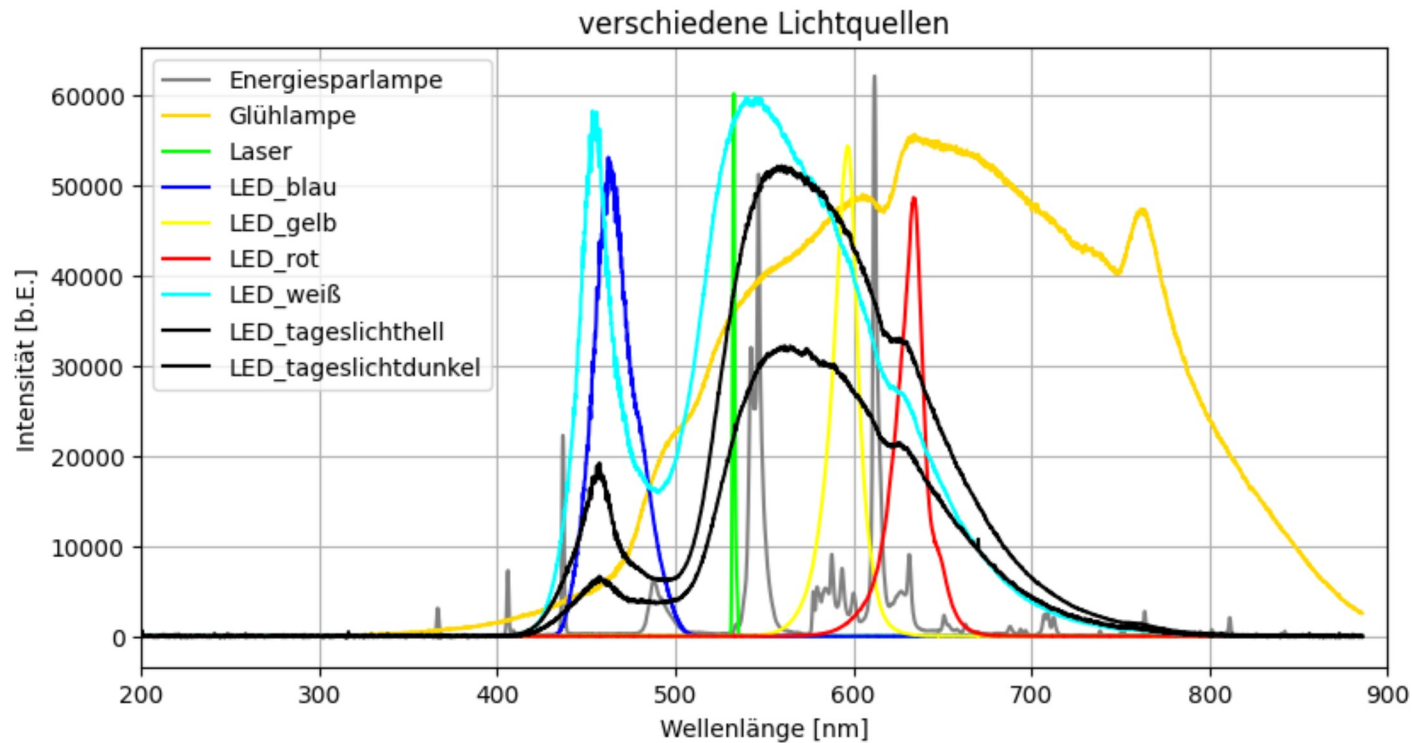
Nr.2 verschiedenen Lichtquellen

```
In [11]: # Lichtquellen-Liste mit entsprechender Farbzuteilung
lichtquellen={"Energiesparlampe":"grey", "Glühlampe":"gold", "Laser":"lime", "LED_blau":"blue", "LED_gelb":"yellow", "LED_rot":"red", "LED_weiß":"cyan", "LED_tageslichthell":

plt.figure(figsize=figsize_cm(12,6))
plt.title('verschiedene Lichtquellen')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.xlim((200,900))
for lichtquelle,color in lichtquellen.items():
    lambda_lichtquelle,intensity_lichtquelle = np.loadtxt(Messwerte_path/"Nr.2"/f"{lichtquelle}.txt",skiprows=17,
    converters= {0:comma_to_float, 1:comma_to_float},
    comments='>', unpack=True)
    plt.plot(lambda_lichtquelle, intensity_lichtquelle,label=lichtquelle,color=color)
    Lambda_lichtquelle=np.array(lambda_lichtquelle)
    Intensity_lichtquelle=np.array(intensity_lichtquelle)
    meanlambda=round(np.sum(Lambda_lichtquelle*Intensity_lichtquelle)/np.sum(Intensity_lichtquelle),2)
    print(f"{lichtquelle}&\num{{{meanlambda}}}\n")
plt.legend()
plt.grid()
# plt.savefig(Ausgabe_path/"Nr.2_Lichtquellen.png", format="png")
```

```
Energiesparlampe&\num{573.91}\\
Glühlampe&\num{658.13}\\
Laser&\num{534.54}\\
LED_blau&\num{467.16}\\
LED_gelb&\num{594.17}\\
LED_rot&\num{630.93}\\
LED_weiß&\num{553.64}\\
LED_tageslichthell&\num{581.49}\\
LED_tageslichtdunkel&\num{586.92}\\
```

Figure

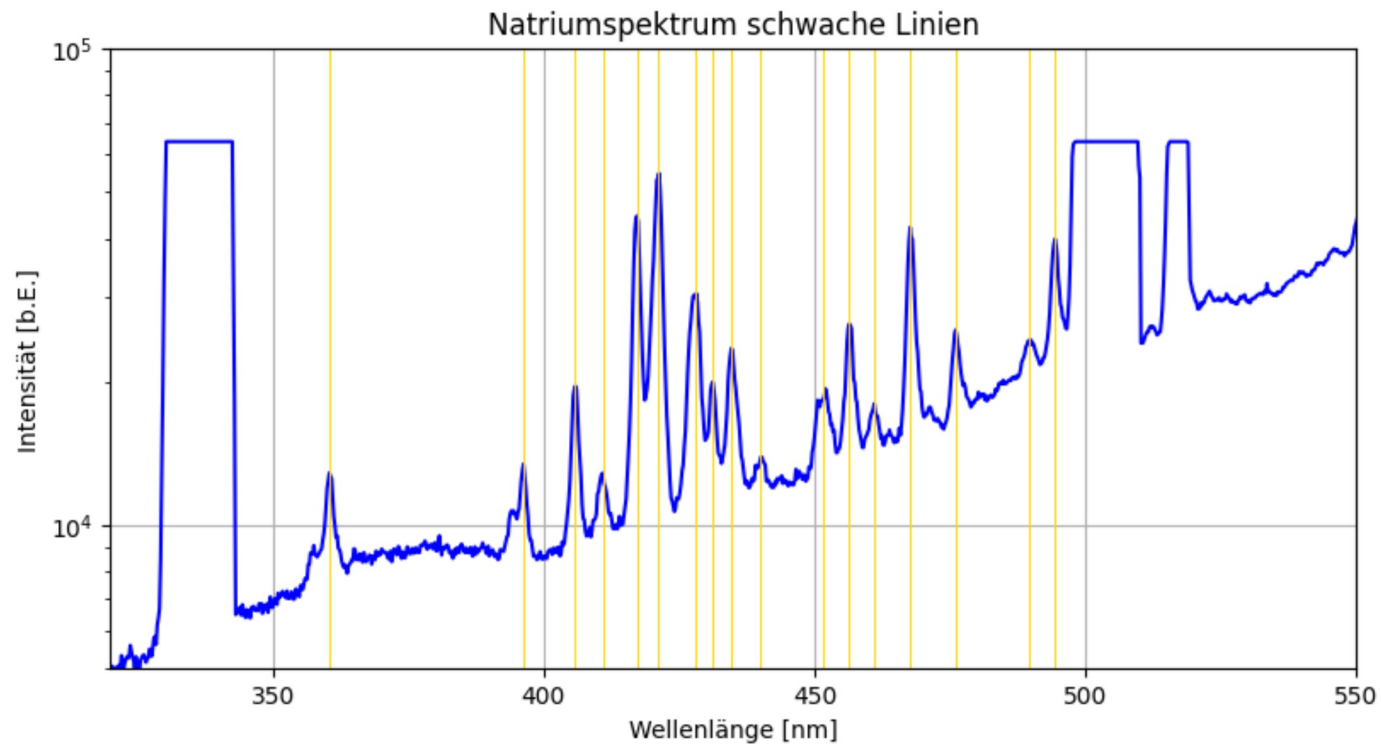


Hier geht der Abfuck los: für jedes aufgenommene Natriumspektrum wird ein Plot gezeichnet, und dann wurden per Hand die Emissionslinien gesucht, dies wurde eigentlich obsolet, da die Plots unten nochmal mit FWHM eingezeichnet wurden.

```
In [24]: lambda_natriumklein,intensity_natriumklein = np.loadtxt(Messwerte_path/"Nr.3"/"Natriumdampflampe_kleinelinie.txt",skiprows=17,
converters={0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)
color = 'blue'

natrium1klein = [360.5,396.3,405.8,411,417.2,421.2,428,431.2,434.6,440.1,451.7,456.4,461,467.6,476,489.7,494.3]
plt.figure(figsize=figsize_cm(12,6))
plt.title('Natriumspektrum schwache Linien')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.yscale('log')
plt.ylim((5000,100000))
plt.xlim((320,550))
plt.plot(lambda_natriumklein, intensity_natriumklein,color=color)
plt.vlines(x=natrium1klein,ymin=0,ymax=100000, colors='gold', linewidths = 0.7)
plt.grid()
plt.savefig(Ausgabe_path/"Nr.3_Natriumspektrumkleinelinie.png", format="png")
```

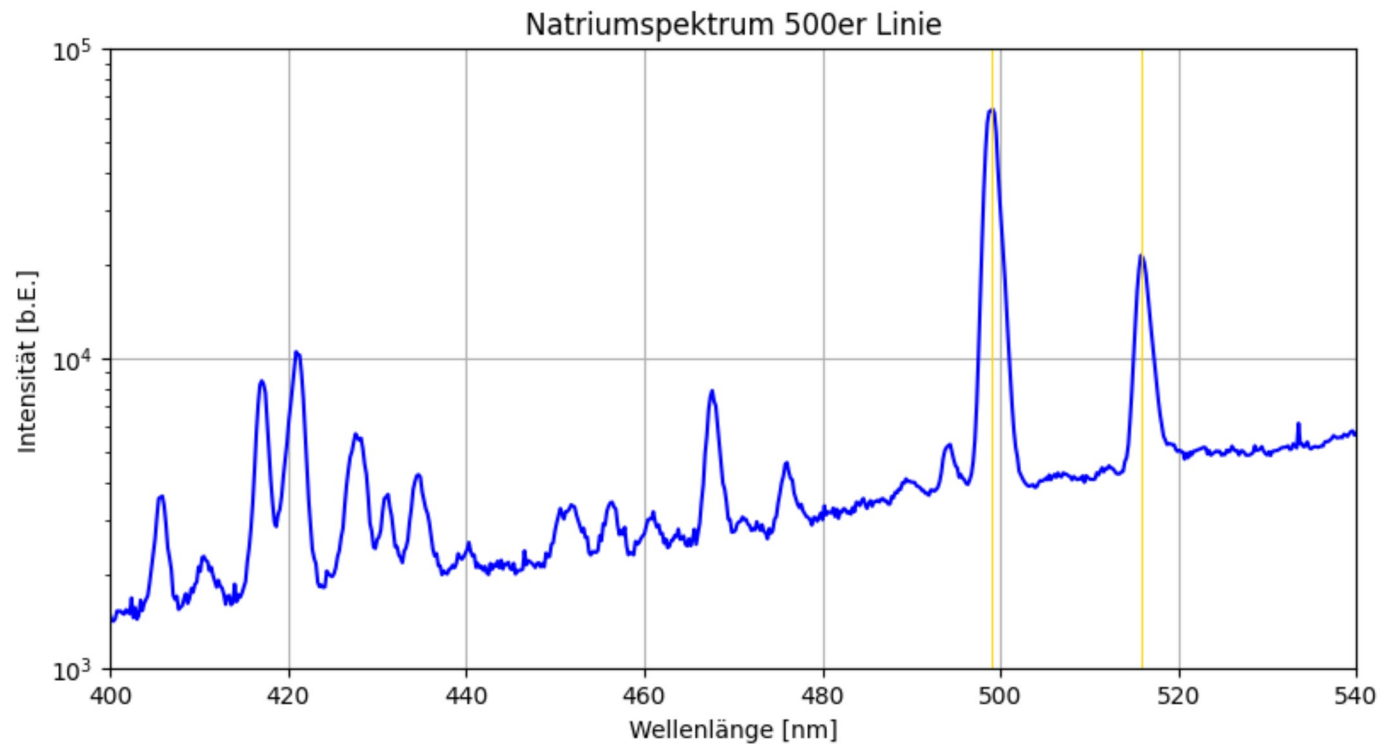
Figure



```
In [11]: lambda_natrium500,intensity_natrium500 = np.loadtxt(Messwerte_path/"Nr.3"/"Natriumdampflampe500linie.txt",skiprows=17,
converters= {0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)

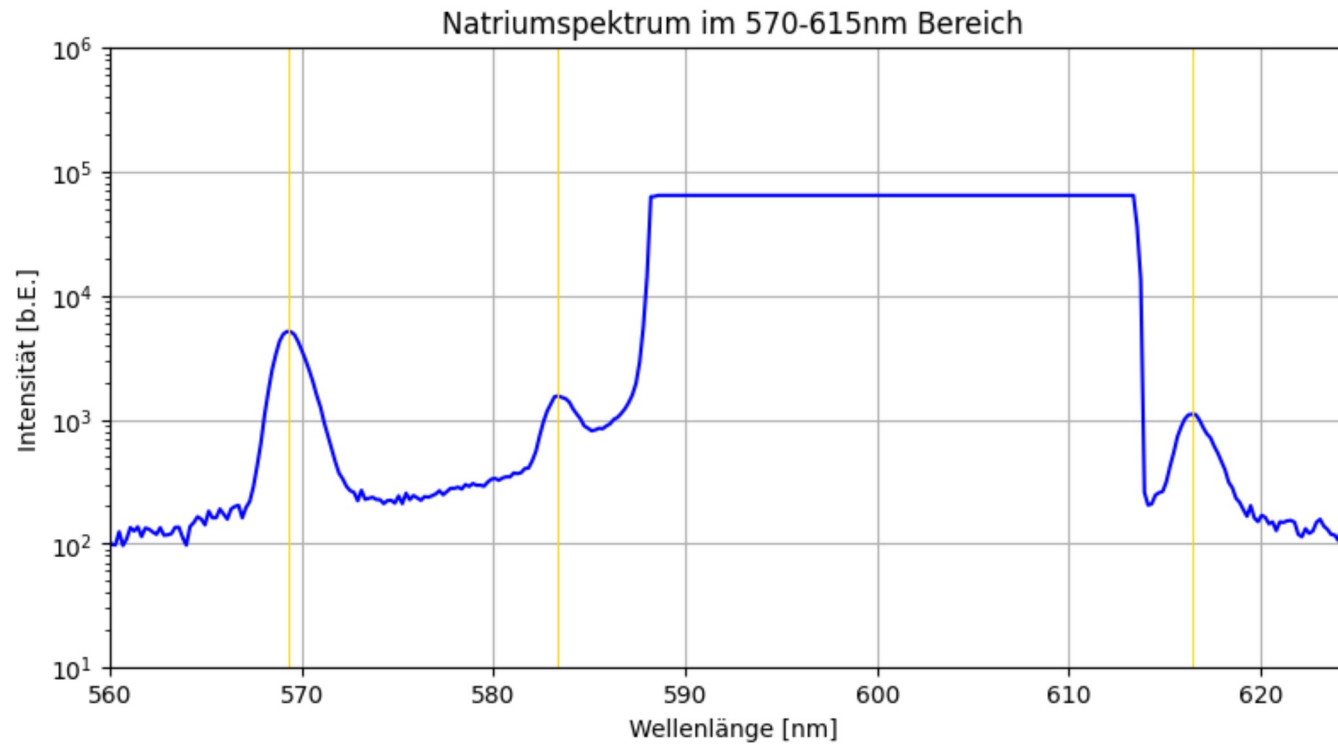
Linien=[499,515.8]
plt.figure(figsize=figsize_cm(12,6))
plt.title('Natriumspektrum 500er Linie')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.yscale('log')
plt.ylim((1000,100000))
plt.xlim((400,540))
plt.plot(lambda_natrium500, intensity_natrium500,color=color)
plt.vlines(x=Linien,ymin=0,ymax=100000, colors='gold', linewidths = 0.7)
plt.grid()
plt.savefig(Ausgabe_path/"Nr.3_Natriumspektrum_500Linie.png", format="png")
```


Figure



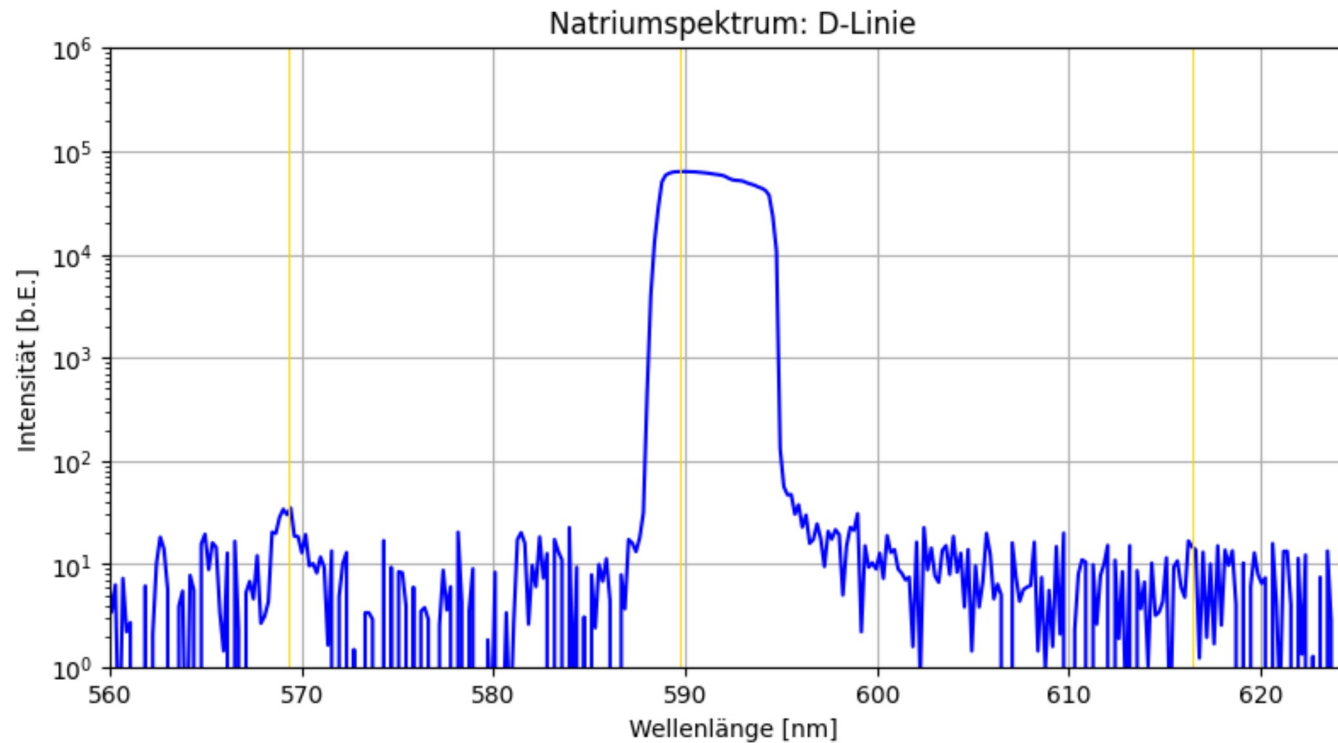
```
In [12]: lambda_natriumD,intensity_natriumD = np.loadtxt(Messwerte_path/"Nr.4"/"natrium570-615.txt",skiprows=17,
converters= {0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)
```

```
Linien=[569.3,583.3,616.4]
plt.figure(figsize=figsize_cm(12,6))
plt.title('Natriumspektrum im 570-615nm Bereich')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.yscale('log')
plt.ylim((10,1000000))
plt.xlim((560,625))
plt.plot(lambda_natriumD, intensity_natriumD,color=color)
plt.vlines(x=Linien,ymin=0,ymax=1000000, colors='gold', linewidths = 0.7)
plt.grid()
plt.savefig(Ausgabe_path/"Nr.3_Natriumspektrum_570-615Linien.png", format="png")
```

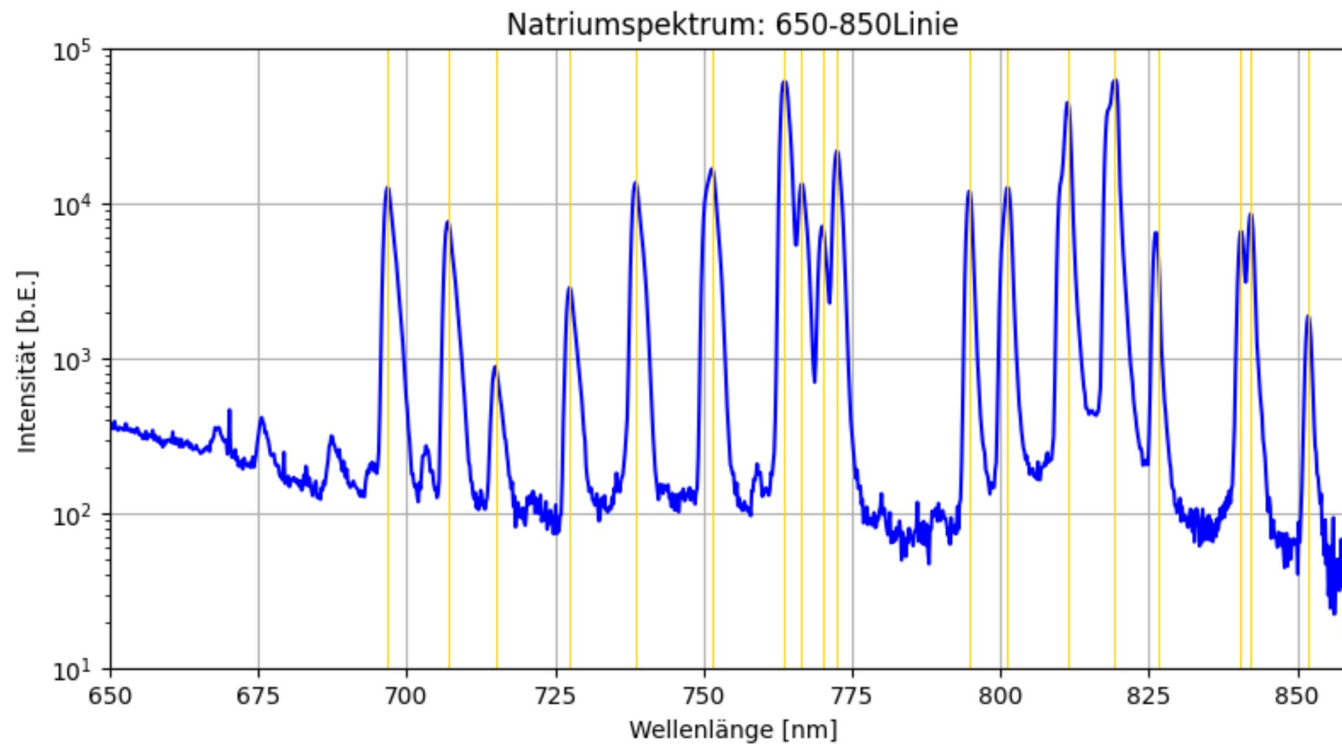
```
In [13]: lambda_natrium570_615,intensity_natrium570_615 = np.loadtxt(Messwerte_path/"Nr.4"/"NatriumD-Linie.txt",skiprows=17,
converters= {0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)
```

```
Linien=[569.3,616.4,589.7]
plt.figure(figsize=figsize_cm(12,6))
plt.title('Natriumspektrum: D-Linie')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.yscale('log')
plt.ylim((1,1000000))
plt.xlim((560,625))
plt.plot(lambda_natrium570_615, intensity_natrium570_615,color=color)
plt.vlines(x=Linien,ymin=0,ymax=1000000, colors='gold', linewidths = 0.7)
plt.grid()
plt.savefig(Ausgabe_path/"Nr.3_Natriumspektrum_D-Linie.png", format="png")
```



```
In [14]: lambda_natrium650_850,intensity_natrium650_850 = np.loadtxt(Messwerte_path/"Nr.5"/"Natrium650-850.txt",skiprows=17,
converters= {0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)

Linien=[696.7,706.9,715,727.5,738.5,751.4,763.6,766.5,770,772.5,794.7,801.2,811.3,819.3,826.6,840.5,842.2,851.8]
plt.figure(figsize=figsize_cm(12,6))
plt.title('Natriumspektrum: 650-850Linie')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.yscale('log')
plt.ylim((10,100000))
plt.xlim((650,860))
plt.plot(lambda_natrium650_850, intensity_natrium650_850,color=color)
plt.vlines(x=Linien,ymin=0,ymax=100000, colors='gold', linewidths = 0.7)
plt.grid()
plt.savefig(Ausgabe_path/"Nr.3_Natriumspektrum_650-850.png", format="png")
```



Ab hier kein Bock mehr, jetzt sollten auch noch FWHM als Fehler gesucht werden smh fuck me

Das war der Anfang vom Kochen mit ChatGPT:

- funktioniert nur für eine Datei.
- Linien sind vorgegeben
- FWHM wird berechnet durch klicken
- speichert nach Stop im Ordner der jpynb-Datei als peak_results.txt. Also keine Differenzierung nach Graph, sondern nur ein einziger fester Titel

```
In [4]: # -----
# Messdaten laden
# -----
lambda_natrium570_615, intensity_natrium570_615 = np.loadtxt(
    Messwerte_path/"Nr.4"/"NatriumD-Linie.txt",
    skiprows=17,
    converters={0:comma_to_float, 1:comma_to_float},
    comments='>',
    unpack=True
)

Linien = [569.3, 616.4, 589.7]
```

```

# -----
# Figure und Plot
# -----
fig, ax = plt.subplots(figsize=figsize_cm(12,6))
plt.subplots_adjust(bottom=0.2) # Platz für Button

ax.set_title('Natriumspektrum: D-Linie')
ax.set_xlabel('Wellenlänge [nm]')
ax.set_ylabel('Intensität [b.E.]')
ax.set_yscale('log')
ax.set_ylim((1,1000000))
ax.set_xlim((560,625))

ax.plot(lambda_natrium570_615, intensity_natrium570_615, color='blue')
ax.vlines(x=Linien, ymin=1, ymax=1000000, colors='gold', linewidths=0.7)
ax.grid()

# -----
# Variablen
# -----
clicks = []
results = []

# -----
# Datei speichern
# -----
def save_results_to_file(filename="peaks_results.txt"):
    with open(filename, "w") as f:
        for res in results:
            # Rundung auf eine Nachkommastelle
            peak = round(res['λ_peak']/2, 1)
            fwhm = round(res['FWHM'], 1)
            line = f"\\num{{{peak}}}{fwhm}} \\\\n"
            f.write(line)

# -----
# STOP-Button
# -----
def stop(event):
    # Button rot färben
    btn_stop.color = 'red'
    btn_stop.hovercolor = 'red'
    btn_stop.label.set_color('white')
    fig.canvas.draw_idle()

    save_results_to_file()
    # plt.close(fig) # optional Plot schließen

ax_stop = plt.axes([0.8, 0.01, 0.1, 0.05])
btn_stop = Button(ax_stop, 'STOP', color='lightgray', hovercolor='gray')
btn_stop.on_clicked(stop)

# -----
# Klick-Event für Peaks

```

```

# -----
def onclick(event):
    if event.inaxes != ax or event.xdata is None:
        return

    clicks.append((event.xdata, event.ydata))

    if len(clicks) == 2:
        lambda_peak = clicks[0][0]
        I2 = clicks[0][1]
        I1=0
        I_half = (I2 - I1)/2

        # Peak-Index im Array
        peak_index = np.argmin(np.abs(lambda_natrium570_615 - lambda_peak))

        # Links vom Peak den Halbwertpunkt finden
        i = peak_index
        while i > 0 and intensity_natrium570_615[i] > I_half:
            i -= 1
        lambda1 = lambda_natrium570_615[i]

        # rechts vom Peak den Halbwertpunkt finden
        i = peak_index
        while i < len(intensity_natrium570_615)-1 and intensity_natrium570_615[i] > I_half:
            i += 1
        lambda2 = lambda_natrium570_615[i]

        fwhm = lambda2 - lambda1

        # Ergebnisse speichern
        results.append({
            'λ_peak': lambda_natrium570_615[peak_index],
            'λ1': lambda1,
            'λ2': lambda2,
            'FWHM': fwhm,
            'I_half': I_half
        })

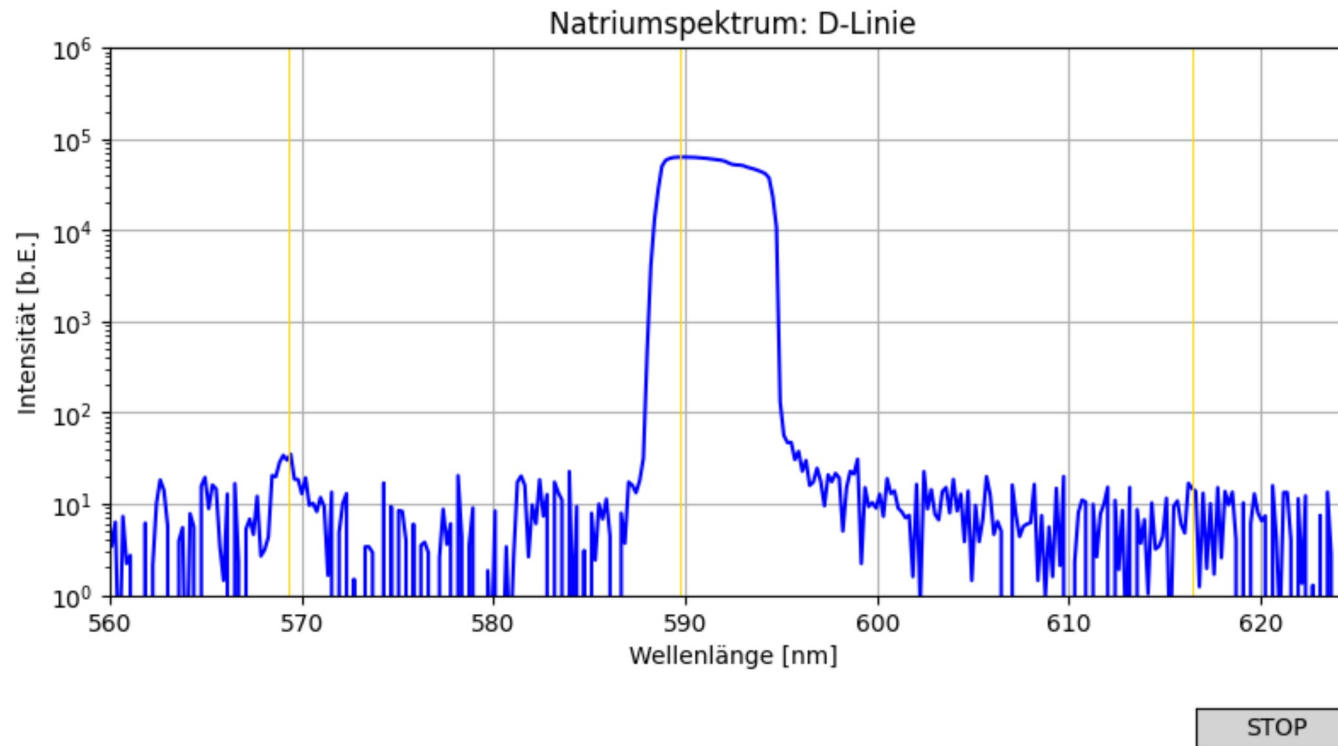
        # Linien zeichnen
        ax.hlines(I_half, xmin=lambda1, xmax=lambda2, color="red", linestyle="--")
        ax.vlines(x=lambda1, ymin=I1, ymax=I2, color="green")
        ax.vlines(x=lambda2, ymin=I1, ymax=I2, color="green")
        fig.canvas.draw_idle()

    clicks.clear() # bereit für nächsten Peak

fig.canvas.mpl_connect("button_press_event", onclick)
plt.show()

```


Figure



Das ist die finale Version für meine Anwendung. Es wird eine globale Funktion `peak_analysis` definiert.

- in graphs werden die Einstellungen zum Laden der 5 Diagramme vorgegeben
- nur FWHM wird eingezeichnet
- Werte werden gespeichert, unterm `TeX_results` Ordner als .txt Datei, je nachdem welche Messreihe man gerade analysiert.

```
In [ ]: # -----
# Messdaten laden
# -----

graphs = [
    {
        "datei": Messwerte_path/"Nr.3"/"Natriumdampflampe_kleinelinie.txt",      # Dateipfad der Messwerte des Gitterspektrometers
        "Messreihe": "kleineLinien",                                           # Name der Messreihe (ist nicht derselbe Name wie der der Messwerte Datei), wird sp
        "xlim": (350, 550),                                                    # Wellenlängebereich der Messreihe
        "ylim": (5000, 100000),                                                # Intensitätsbereich
        "Linien": [360.5,396.3,405.8,411,417.2,421.2,428,431.2,434.6,440.1,451.7,456.4,461,467.6,476,489.7,494.3] # Werte der in den Python Codes von ganz am Anf
    },
    {
        "datei": Messwerte_path/"Nr.3"/"Natriumdampflampe500linie.txt",
        "Messreihe": "500Linien",
        "xlim": (400, 540),
    }
]
```

```

        "ylim": (1000,100000),
        "Linien": [499,515.8]
    },
    {
        "datei": Messwerte_path/"Nr.4"/"natrium570-615.txt",
        "Messreihe": "570-615Linien",
        "xlim": (560,625),
        "ylim": (10,1000000),
        "Linien": [569.3,583.3,616.4]
    },
    {
        "datei": Messwerte_path/"Nr.4"/"NatriumD-Linie.txt",
        "Messreihe": "DLinien",
        "xlim": (560,625),
        "ylim": (1,1000000),
        "Linien": [569.3,616.4,589.7]
    },
    {
        "datei": Messwerte_path/"Nr.5"/"Natrium650-850.txt",
        "Messreihe": "650-850Linien",
        "xlim": (650, 860),
        "ylim": (10, 100000),
        "Linien": [696.7,706.9,715,727.5,738.5,751.4,763.6,766.5,770,772.5,794.7,801.2,811.3,819.3,826.6,840.5,842.2,851.8]
    },
]

def peak_analysis(datei_path, Messreihe, Linien, xlim=(0,1000), ylim=(0,1e6)):
    lambda_values, intensity_values = np.loadtxt(
        datei_path,
        skiprows=17,
        converters={0:comma_to_float, 1:comma_to_float},
        comments='>',
        unpack=True
    )

    results = []

    # ----- Plot -----
    fig, ax = plt.subplots(figsize=(12,6))
    plt.subplots_adjust(bottom=0.2)
    ax.plot(lambda_values, intensity_values, color='blue')
    ax.set_xlim(xlim)
    ax.set_ylim(ylim)
    ax.set_yscale('log')
    ax.set_xlabel("Wellenlänge [nm]")
    ax.set_ylabel("Intensität [b.E.]")
    ax.set_title(f"Natriumspektrum: {Messreihe}")
    ax.vlines(x=Linien, ymin=ylim[0], ymax=ylim[1], colors='gold', linewidths=0.7)
    ax.grid()

    clicks = []

    ax_btn_stop = plt.axes([0.7, 0.01, 0.2, 0.05])

```

*# die Argumente entsprechen den Einträgen der graphs-items (Einstellungen). Übergeb
Laden der Spalten aus der Gitterdatei in zwei Arrays, die dann zum Plotten der I(*

erstellen der results-List, in der die Ergebnisse für TeX_results gespeichert wer

erstellen der figure (fig) und der Achsen (ax)

*# I(\lambda) (zwei Spalten aus der Gitterdatei) werden geplottet
laden des ylim-Arguments von peak_analysis
""*

*# Plottitel anhand des Messreihen arguments von peak_analysis wird festgelegt
Emissionslinien werden geplottet, vom 0.Argument von ylim bis zum 1. Argument von*

erstellen der clicks Liste, in der die FWHM Klicks gespeichert werden

Positionieren des Buttons relativ zu den Plot-Achsen

```

btn_stop = Button(ax_btn_stop, 'Press to stop measuring', color='lightgray', hovercolor='gray') # Erstellen des Buttons btn_stop

# ----- Klick-Event ----- # Klicken zur Berechnung/Einzeichnung der FWHM
def onclick(event):
    if event.button != 1: # nur Linksklick
        return
    if event.inaxes != ax or event.xdata is None:
        return
    # Toolbar-Interaktion ignorieren (Zoom oder Pan)
    if plt.get_current_fig_manager().toolbar.mode != '':
        return

    clicks.append((event.xdata, event.ydata)) # füllen der clicks Liste mit den Positionsdaten des Mauszeigers ('click'-event.xdata, event.ydata)
    if len(clicks) == 2: # nach zweimal drücken (also auf dem Untergrund des Peaks und dem Peak selber) wird
        lambda_peak = clicks[1][0] # clicks[1][0] nimmt aus dem ersten clicks-item (xdata, ydata) das nullte Element,
        I1, I2 = clicks[0][1], clicks[1][1] # da im Plot kontinuierliche Werte angezeigt werden, wird der nächste Datenpunkt auf
        I_half = I1 + (I2 - I1)/2
        peak_index = np.argmax(np.abs(lambda_values - lambda_peak)) # für 0 < lambda < lambda_peak bis zu I(lambda)=I_half läuft die Schleife

        # suchen nach der linken Wellenlänge der FWHM
        i = peak_index
        while i > 0 and intensity_values[i] > I_half: # lambda_1 (links) wurde gefunden
            i -= 1
        lambda1 = lambda_values[i]

        # suchen nach der rechten Wellenlänge der FWHM
        i = peak_index
        while i < len(intensity_values)-1 and intensity_values[i] > I_half: # für lambda > lambda_peak < maxLambda der Datei und I(lambda) > I_half wird gesucht
            i += 1
        lambda2 = lambda_values[i] # lambda_2 (rechts) wurde gefunden

        fwhm = lambda2 - lambda1 # berechnen der FWHM

        results.append({ # eintragen der Rechenwerte in die results Liste
            'lambda_peak': lambda_values[peak_index],
            'lambda1': lambda1,
            'lambda2': lambda2,
            'FWHM': fwhm,
            'I_half': I_half
        })

        ax.hlines(I_half, xmin=lambda1, xmax=lambda2, color="red") # Zeichnen der FWHM
        ax.vlines(x=lambda1, ymin=I1, ymax=I_half, color="green")
        ax.vlines(x=lambda2, ymin=I1, ymax=I_half, color="green")
        fig.canvas.draw_idle() # keine Ahnung was das macht, irgendwie timet dies das Neuzeichnen der Grafik, wenn

        clicks.clear() # es wird immer nur ein Klick gespeichert, sodass man mit harten Zahlen bei I_1=cli

# ----- Speichern -----
def save_results_to_file(filename=f"Natrium_{Messreihe}_TeXresults.txt"):
    filepath = base/"TeX_results"/ filename
    with open(filepath, "w") as f:
        for res in results:
            peak = round(res['lambda_peak'], 1)
            # speichern der TeX_results Datei, anhand der Messreihen-Einstellung wird der filename festgelegt,
            # anhand des filenames wird der Speicherpfad festgelegt, relativ zu base (wurde am ...
            # results.txt wird als f geöffnet
            # für alle Items in der results Liste
            # werden die zwei in der .write benötigten Variablen benannt und mit den results Li

```

```

        fwhm = round(res['FWHM']/2, 1)
        f.write(f"\num{{{peak}}({fwhm})} \\\n")

# ----- STOP-Button -----
def stop(event):
    fig.canvas.mpl_disconnect(cid)
    btn_stop.color = 'red'
    btn_stop.hovercolor = 'red'
    btn_stop.label.set_text('Measuring done')
    fig.canvas.draw_idle()
    save_results_to_file()
    # plt.close(fig) # optional
    fig.savefig(Ausgabe_path/f"Nr.3_{Messreihe}_FWHM.png", format="png")

btn_stop.on_clicked(stop)
cid = fig.canvas.mpl_connect("button_press_event", onclick)
plt.show()

```

für alle Einträge in der results Liste wird eine Zeile in die results.txt geschri

wenn man mit auswählen aller peaks fertig ist, stop drücken, danach wird alles ge

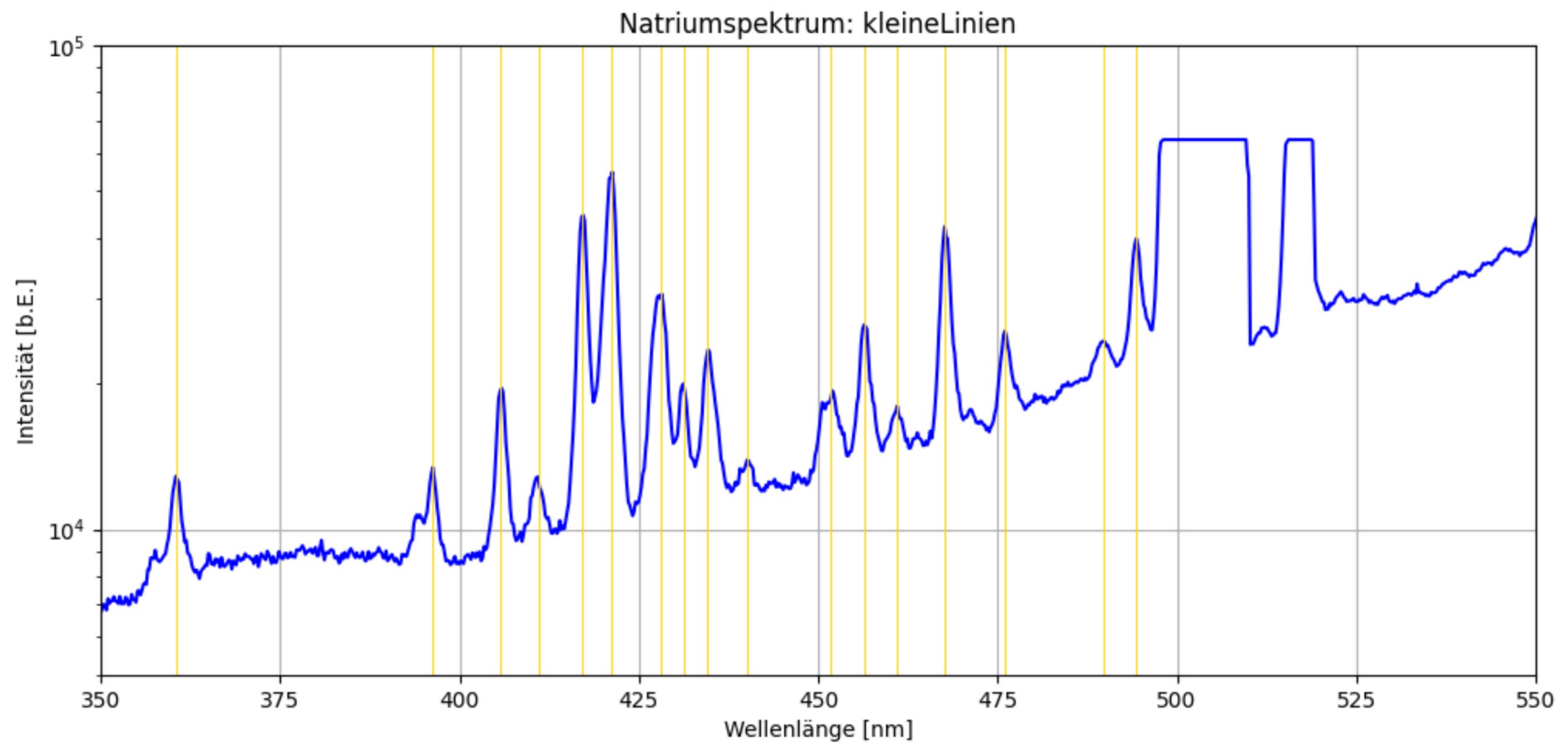
wird der Button gedrückt (also die .on_clicked-Aktion), wird die Stop Funktion au

bei jedem Klick wird neu gezeichnet

```

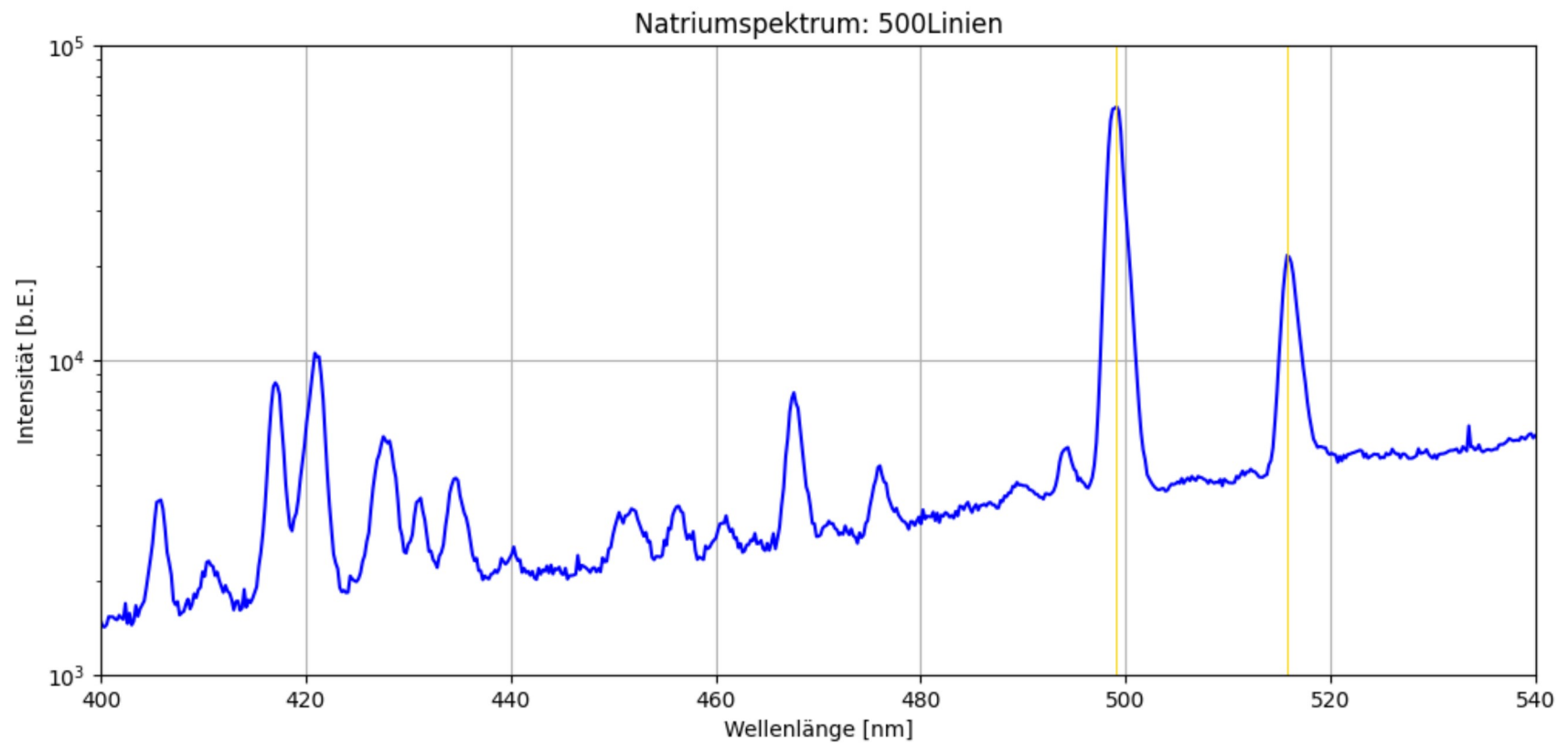
In [14]: index_graph = 0
peak_analysis(graphs[index_graph]["datei"], graphs[index_graph]["Messreihe"], graphs[index_graph]["Linien"], graphs[index_graph]["xlim"], graphs[index_graph]["ylim"])

```



```
In [15]: index_graph = 1  
peak_analysis(graphs[index_graph]["datei"], graphs[index_graph]["Messreihe"], graphs[index_graph]["Linien"], graphs[index_graph]["xlim"], graphs[index_graph]["ylim"])
```

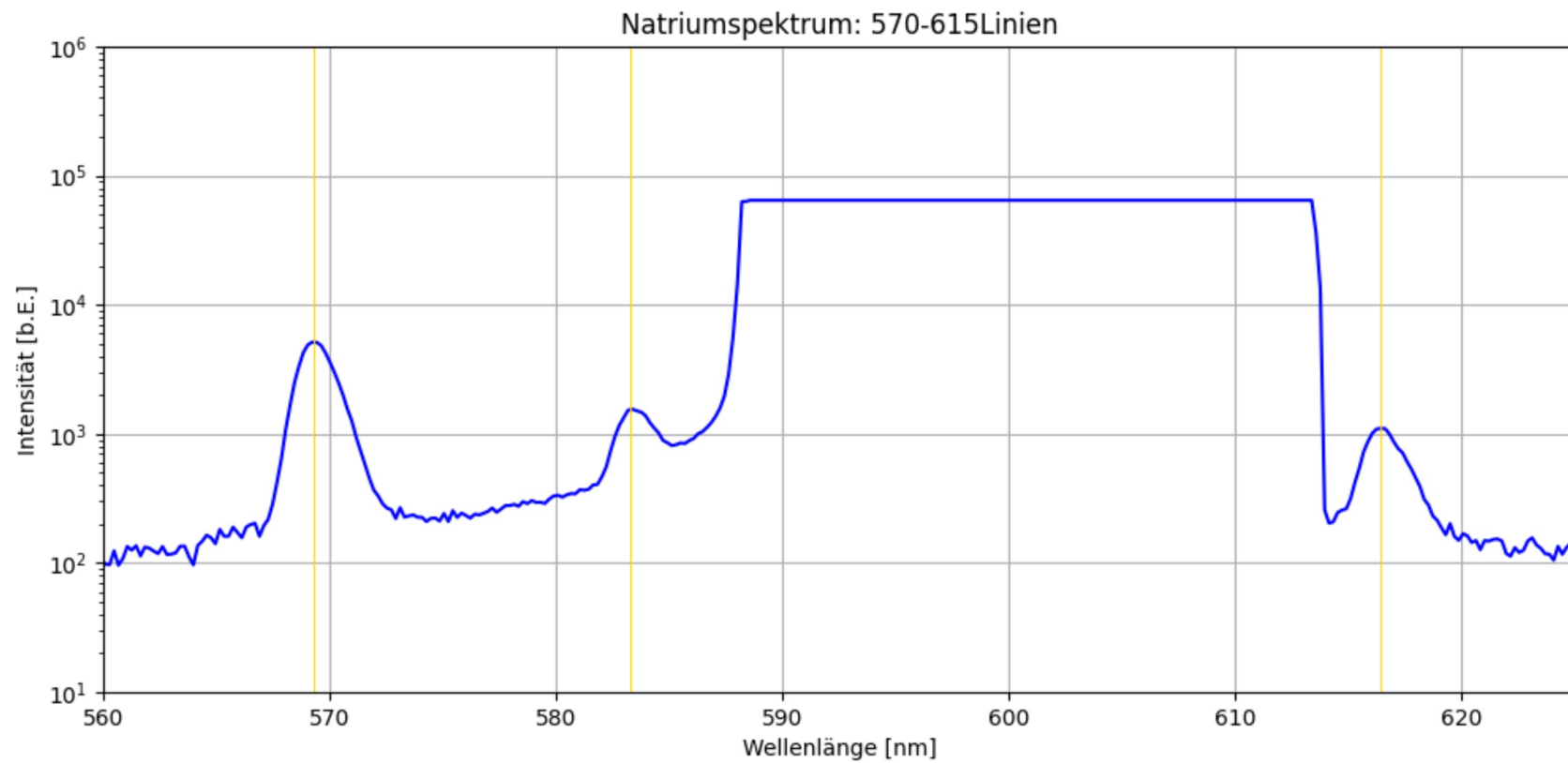
Figure



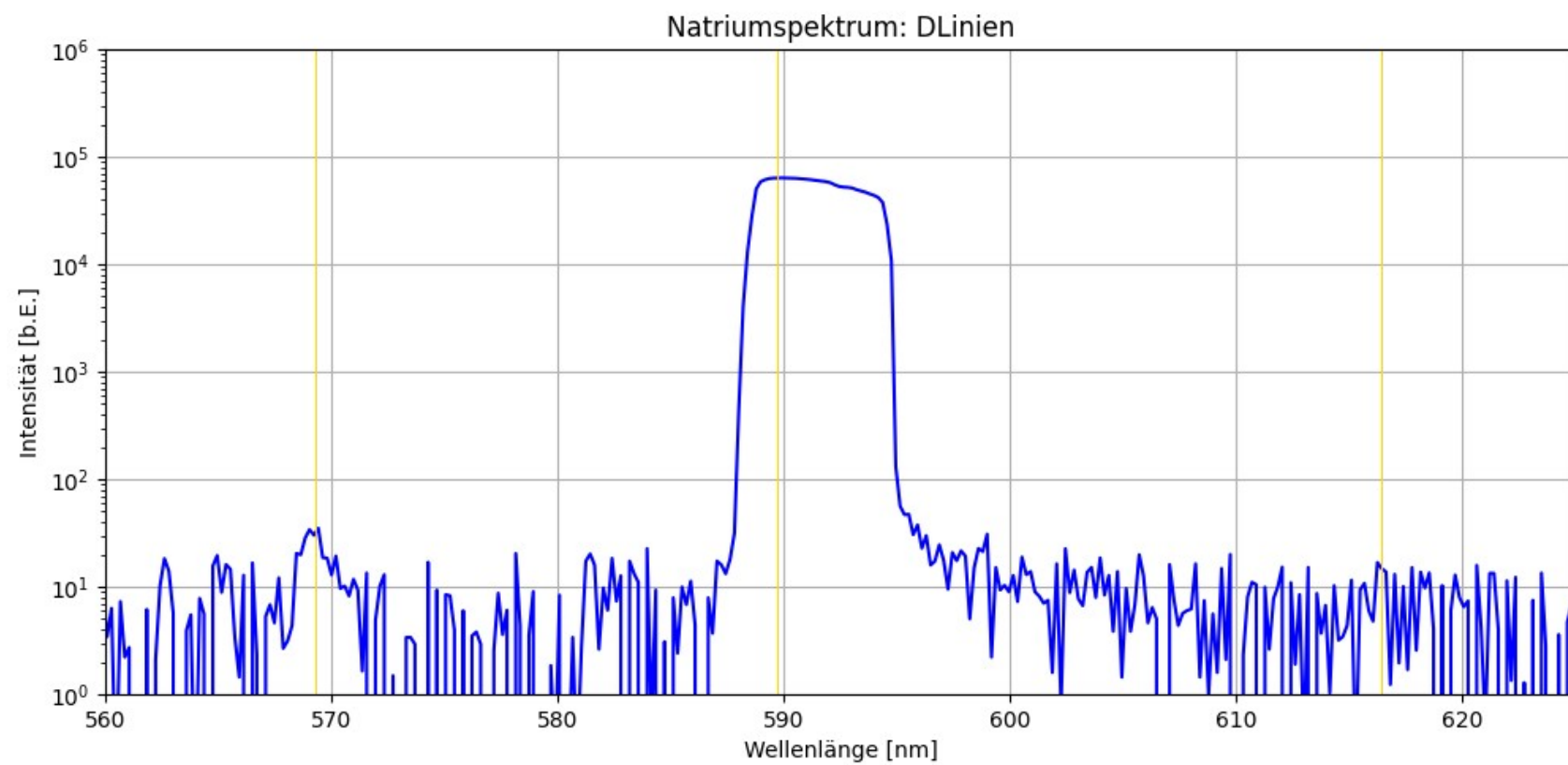
Press to stop measuring

```
In [16]: index_graph = 2
peak_analysis(graphs[index_graph]["datei"], graphs[index_graph]["Messreihe"], graphs[index_graph]["Linien"], graphs[index_graph]["xlim"], graphs[index_graph]["ylim"])
```

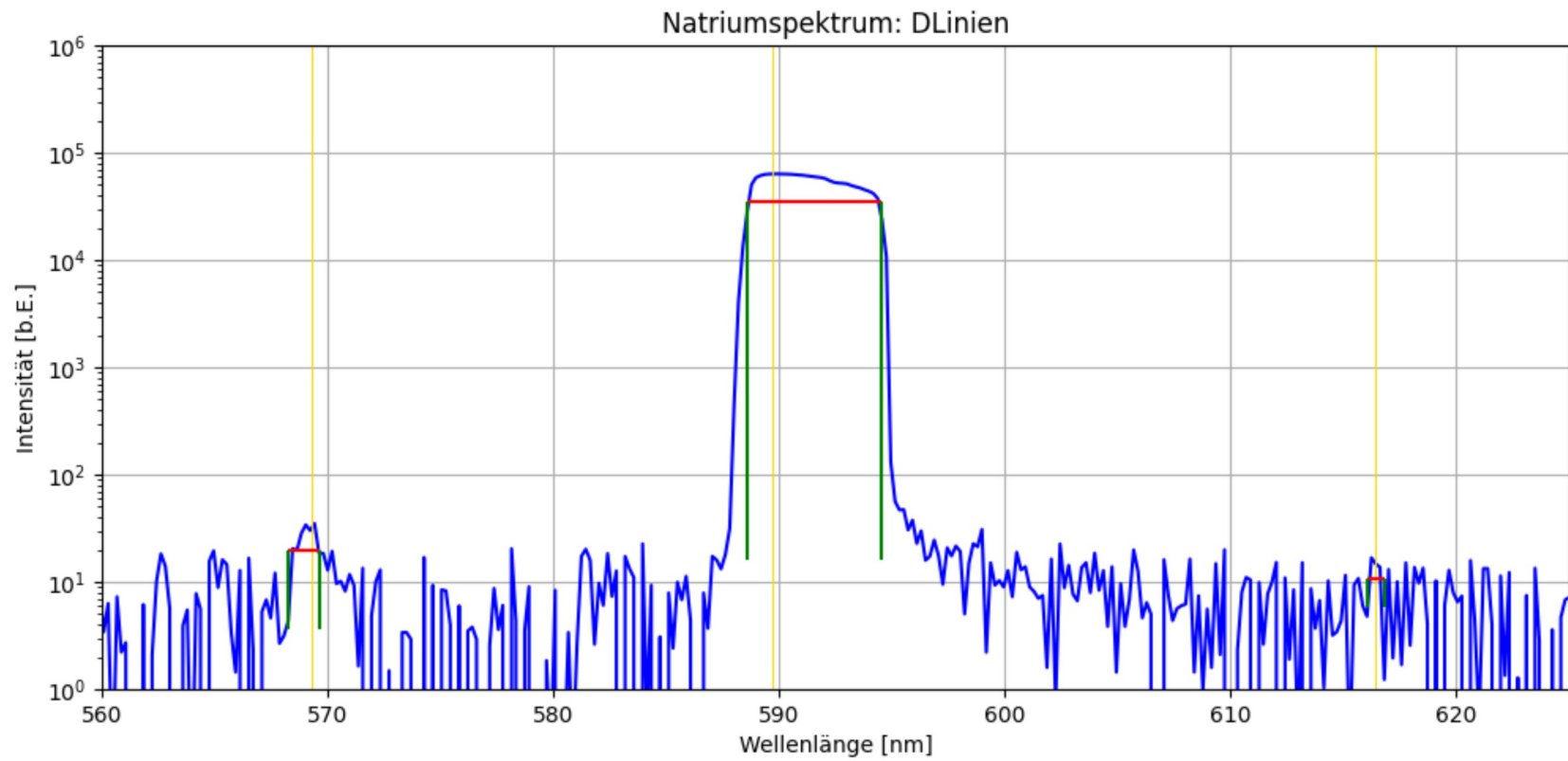

Figure



```
In [17]: index_graph = 3
peak_analysis(graphs[index_graph]["datei"], graphs[index_graph]["Messreihe"], graphs[index_graph]["Linien"], graphs[index_graph]["xlim"], graphs[index_graph]["ylim"])
```

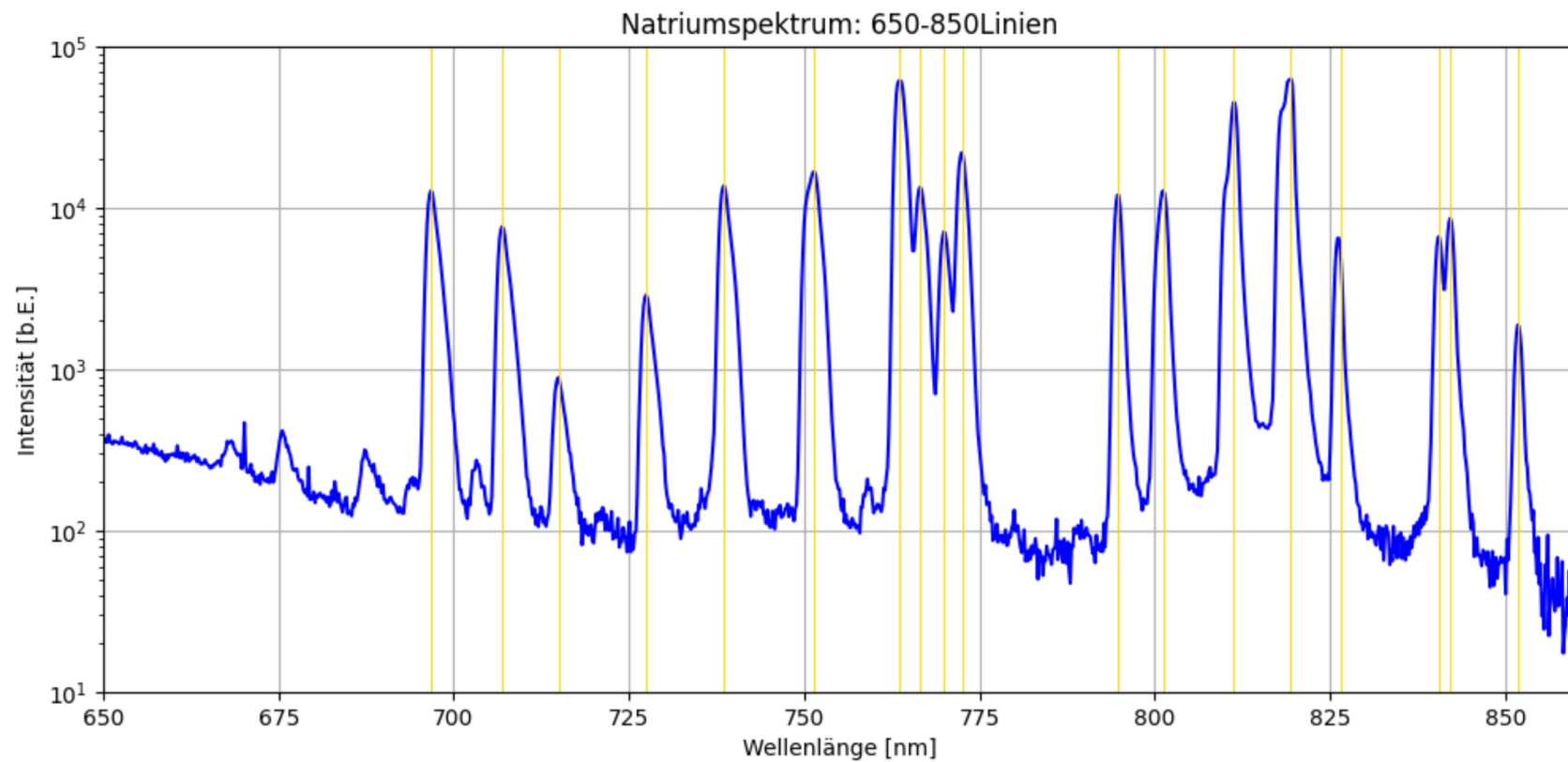


Press to stop measuring



Measuring done

```
In [18]: index_graph = 4
peak_analysis(graphs[index_graph]["datei"], graphs[index_graph]["Messreihe"], graphs[index_graph]["Linien"], graphs[index_graph]["xlim"], graphs[index_graph]["ylim"])
```



Press to stop measuring

Nr.3 Berechnen der Nebenserien

Erwartete Linien für die 1. Nebenserie: $md \rightarrow 3p$

```
In [2]: #Berechnen von E_3p
RyE = -1*c.h*c.c.c.Rydberg/c.eV
m = 3
lamb_3 = 819
Delta_lamb_3 = 1.5
E_3p = RyE/ m**2 - c.h * c.c / c.eV / (lamb_3*1e-9)
Delta_E_3p = c.h * c.c / c.eV / (lamb_3*1e-9)**2 * Delta_lamb_3*1e-9
print(f'E_3p={round(E_3p,4)}, {round(Delta_E_3p,4)}')
```

```
def lamb_m(m):
    lamb = c.h * c.c / c.eV / (RyE/ m**2 - E_3p)*1e9
    Delta_lamb = c.h * c.c * Delta_E_3p * c.eV / (RyE * c.eV / m**2 - E_3p * c.eV)**2 * 1e9
```

```

    return lamb, Delta_lamb

nebenserie1 = []
for m in range(3,14):
    l, k = lamb_m(m)
    nebenserie1.append((l,k))
    print(f'm={m:2d}, lambda = {round(nebenserie1[m-3][0],1)} ± {round(nebenserie1[m-3][1],1)}')

for m in range(3,14):
    print(f'{m}&\num{{{round(nebenserie1[m-3][0],1)}({round(nebenserie1[m-3][1],1)})}}')

```

```

E_3p=-3.0256, 0.0028
m= 3, lambda = 819.0 ± 1.5
m= 4, lambda = 570.0 ± 0.7
m= 5, lambda = 499.7 ± 0.6
m= 6, lambda = 468.3 ± 0.5
m= 7, lambda = 451.2 ± 0.5
m= 8, lambda = 440.8 ± 0.4
m= 9, lambda = 433.9 ± 0.4
m=10, lambda = 429.1 ± 0.4
m=11, lambda = 425.6 ± 0.4
m=12, lambda = 423.0 ± 0.4
m=13, lambda = 421.0 ± 0.4
3&\num{819.0(1.5)}
4&\num{570.0(0.7)}
5&\num{499.7(0.6)}
6&\num{468.3(0.5)}
7&\num{451.2(0.5)}
8&\num{440.8(0.4)}
9&\num{433.9(0.4)}
10&\num{429.1(0.4)}
11&\num{425.6(0.4)}
12&\num{423.0(0.4)}
13&\num{421.0(0.4)}

```

Erwartete Linien für die 2. Nebenserie: $m_s \rightarrow 3p$

```

In [10]: lamb_D=589
Delta_lamb_D = 1.5
E_3s = E_3p-c.c*c.h/(c.eV*lamb_D*1e-9)
Delta_E_3s = np.sqrt((Delta_E_3p)**2 + (1.2398e3/(lamb_D)**2 * (Delta_lamb_D))**2)
print(f'E_3s={round(E_3s,3)}, {round(Delta_E_3s,4)}')

korrs = 3-np.sqrt(RyE/E_3s)
Delta_korrs =np.sqrt(RyE/E_3s**3)*Delta_E_3s/2
print(f'korrs={round(korrs,4)}, {round(Delta_korrs,4)}')

def lamb_m(m):
    lamb = c.h * c.c / c.eV / (RyE/ (m-korrs)**2 - E_3p)*1e9
    Delta_lamb = np.sqrt((((2*RyE*c.c*c.eV*c.h/((-korrs + m)**3*(-E_3p*c.eV+RyE*c.eV/((-korrs + m)**2)**2))*Delta_korrs)**2 + ((c.c*c.eV*c.h/(-E_3p*c.eV+ RyE*c.eV/(-korrs + m-korrs)**2 - E_3p)*1e9)**2 - E_3p)**2))**2))
    return lamb, Delta_lamb

nebenserie2 = []
for m in range(4,10):
    l, k = lamb_m(m)

```

```

nebenserie2.append((l,k))
print(f'm={m:2d}, lambda = {round(nebenserie2[m-4][0],1)} ± {round(nebenserie2[m-4][1],1)}')

for m in range(4,10):
    print(f'{m}&\num{{{round(nebenserie2[m-4][0],1)}({round(nebenserie2[m-4][1],1)})}}')

```

```

E_3s=-5.131,0.006
korrs=1.3715,0.001
m= 4, lambda = 1173.8 ± 3.5
m= 5, lambda = 622.4 ± 0.9
m= 6, lambda = 518.7 ± 0.6
m= 7, lambda = 477.6 ± 0.5
m= 8, lambda = 456.5 ± 0.5
m= 9, lambda = 444.1 ± 0.4
4&\num{1173.8(3.5)}
5&\num{622.4(0.9)}
6&\num{518.7(0.6)}
7&\num{477.6(0.5)}
8&\num{456.5(0.5)}
9&\num{444.1(0.4)}

```

Erwartete Linien für die Hauptserie: $mp \rightarrow 3s$

```

In [11]: korrp = 3-np.sqrt(RyE/E_3p)
Delta_korrp =np.sqrt(RyE/E_3p**3)*Delta_E_3p
print(f'korrs={round(korrp,4)}, {round(Delta_korrp,4)}')

def lamb_m(m):
    lamb = c.h * c.c / c.eV / (RyE / (m-korrp)**2 - E_3s)*1e9
    Delta_lamb = np.sqrt((( -2*RyE*c.c*c.eV*c.h/((-korrp + m)**3*(-E_3s*c.eV +RyE*c.eV/(-korrp + m)**2)**2))*Delta_korrp)**2 + ((c.c*c.eV*c.h/(-E_3s*c.eV+ RyE*c.eV/(-ko
    return lamb, Delta_lamb

hauptserie = []
for m in range(4,6):
    l, k = lamb_m(m)
    hauptserie.append((l,k))
    print(f'm={m:2d}, lambda = {round(hauptserie[m-4][0],1)} ± {round(hauptserie[m-4][1],1)}')

for m in range(4,6):
    print(f'{m}&\num{{{round(hauptserie[m-4][0],1)}({round(hauptserie[m-4][1],1)})}}')

korrs=0.8794,0.0019
m= 4, lambda = 332.1 ± 0.6
m= 5, lambda = 286.4 ± 0.4
4&\num{332.1(0.6)}
5&\num{286.4(0.4)}

```

Berechnung der Sigma-Abweichungen die data-Felder sind die zugeordneten Zeilen aus der LateX-Tabelle, in denen die berechneten und aus den Plots extrahierten Emissionslinien von Natrium eingetragen wurden.

```

In [ ]: import re
import math

data1 = """

```

```

3&\num{819.0(1.5)}&\num{819.1(1.1)} \\
4&\num{570.0(0.7)}&\num{569.2(1.0)} \\
5&\num{499.7(0.6)}&\num{498.9(1.0)} \\
6&\num{468.3(0.5)}&\num{467.6(0.9)} \\
7&\num{451.2(0.5)}&\num{452.2(1.7)} \\
8&\num{440.8(0.4)}&\num{439.7(1.1)} \\
9&\num{433.9(0.4)}&\num{434.7(1.0)} \\
10&\num{429.1(0.4)}&\num{428.2(1.4)} \\
13&\num{421.0(0.4)}&\num{421.1(1.1)}&\\
"""

```

```

pattern = r'\num{([\d.]+\s+)(\s+([\d.]+\s+))}'
patternm = r'^(\d+)'

```

```

nebenserie1_wellen=[]
nebenserie1_wellenfehler=[]
nebenserie1m=[]
for line in data1.strip().splitlines():
    matches = re.findall(pattern, line)
    matchm = re.search(patternm, line)
    if len(matches) == 2 and matchm:
        m = int(matchm.group(1))
        val1, err1 = map(float, matches[0])
        val2, err2 = map(float, matches[1])
        nebenserie1_wellen.append(val2)
        nebenserie1_wellenfehler.append(err2)
        nebenserie1m.append(m)
        sigma = abs(val1 - val2) / math.sqrt(err1**2 + err2**2)
        print(f"&{sigma:.2f}")

```

```

Nebenserie1_wellen=np.array(nebenserie1_wellen)
Nebenserie1_wellenfehler=np.array(nebenserie1_wellenfehler)
Nebenserie1m=np.array(nebenserie1m)
print(Nebenserie1m)
print(Nebenserie1_wellen)
print(Nebenserie1_wellenfehler)

```

```

print()

```

```

data2= """
5&\num{622.4(0.9)}&\num{616.2(0.4)} \\
6&\num{518.7(0.6)}&\num{515.8(1.0)} \\
7&\num{477.6(0.5)}&\num{475.8(1.1)} \\
8&\num{456.5(0.5)}&\num{456.0(1.0)} \\
9&\num{444.1(0.4)}&\num{439.7(1.1)} \\
"""

```

```

nebenserie2_wellen=[]
nebenserie2_wellenfehler=[]
nebenserie2m=[]
for line in data2.strip().splitlines():
    matches = re.findall(pattern, line)

```



```

matchm = re.search(patternm, line)
if len(matches) == 2 and matchm:
    m = int(matchm.group(1))
    val1, err1 = map(float, matches[0])
    val2, err2 = map(float, matches[1])
    nebenserie2_wellen.append(val2)
    nebenserie2_wellenfehler.append(err2)
    nebenserie2m.append(m)
    sigma = abs(val1 - val2) / math.sqrt(err1**2 + err2**2)
    print(f"&{sigma:.2f}")

```

```

Nebenserie2_wellen=np.array(nebenserie2_wellen)
Nebenserie2_wellenfehler=np.array(nebenserie2_wellenfehler)
Nebenserie2m=np.array(nebenserie2m)
print(Nebenserie2m)
print(Nebenserie2_wellen)
print(Nebenserie2_wellenfehler)

```

```

&0.05
&0.66
&0.69
&0.68
&0.56
&0.94
&0.74
&0.62
&0.09
[ 3  4  5  6  7  8  9 10 13]
[819.1 569.2 498.9 467.6 452.2 439.7 434.7 428.2 421.1]
[1.1 1.  1.  0.9 1.7 1.1 1.  1.4 1.1]

&6.30
&2.49
&1.49
&0.45
&3.76
[5 6 7 8 9]
[616.2 515.8 475.8 456.  439.7]
[0.4 1.  1.1 1.  1.1]

```

Fitten der gemessenen Ergebnisse aus der Natriumtabelle der ersten Nebenserie

```

In [29]: def fit_func1(m,E_Ry,E_3p,D_d):
          return 1.2398E3/(E_Ry/(m-D_d)**2-E_3p)
          para = [-13.6,-3,-0.02]
          popt, pcov = curve_fit(fit_func1,Nebenserie1m,Nebenserie1_wellen,sigma=Nebenserie1_wellenfehler,p0=para)
          x=np.linspace(2.8,13.2, 100)

          plt.figure(dpi=150)
          plt.errorbar(Nebenserie1m,Nebenserie1_wellen,yerr=Nebenserie1_wellenfehler,fmt=".",capsize=3,elinewidth=1,label='Messwerte')
          plt.plot(x,fit_func1(x,*popt))
          plt.xlabel('Quantenzahl')
          plt.ylabel('Wellenlänge [nm]')
          plt.title('1. Nebenserie des Na-Atoms')
          plt.legend()

```

```

plt.grid()
plt.savefig(Ausgabe_path/"Nr.3_Nebenserie1.png", format="png")

print("E_Ry=",popt[0], ", Standardfehler=", np.sqrt(pcov[0][0]))
print("E_3p=",popt[1], ", Standardfehler=", np.sqrt(pcov[1][1]))
print("D_d=",popt[2], ", Standardfehler=", np.sqrt(pcov[2][2]))

chi2=np.sum((fit_func1(Nebenserie1m,*popt)- Nebenserie1_wellen)**2/ Nebenserie1_wellenfehler**2)
dof=len(Nebenserie1m)-3 #dof:degrees of freedom, Freiheitsgrad
chi2_red=chi2/dof
print("chi2=", chi2_)
print("chi2_red=",chi2_red)

prob=round(1-chi2.cdf(chi2_,dof),2)*100
print("Wahrscheinlichkeit:", prob,"%")

```

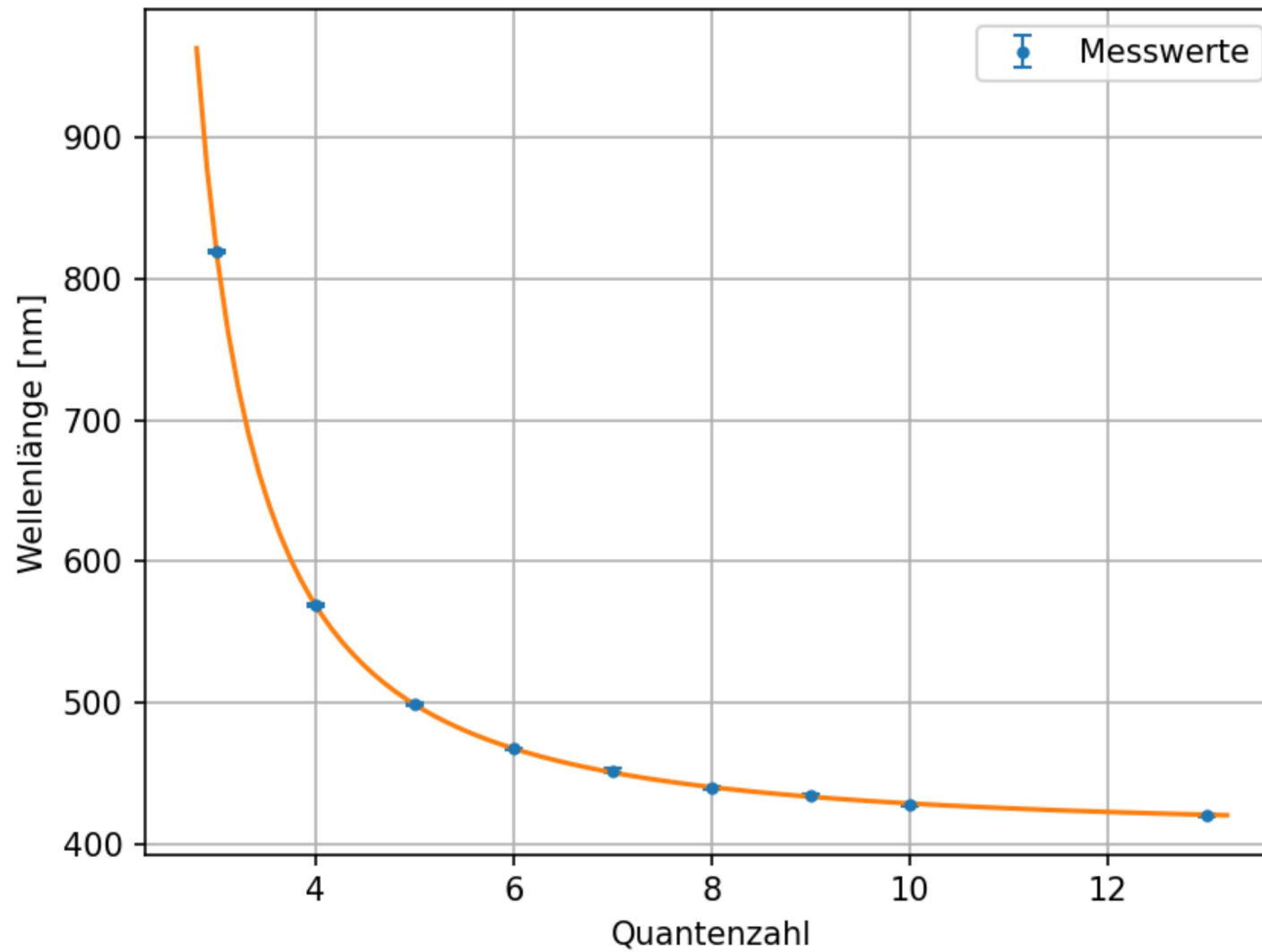
C:\Users\samue\AppData\Local\Temp\ipykernel_21880\988458060.py:7: RuntimeWarning: More than 20 figures have been opened. Figures created through the pyplot interface (`matplotlib.pyplot.figure`) are retained until explicitly closed and may consume too much memory. (To control this warning, see the rcParam `figure.max_open_warning`). Consider using `matplotlib.pyplot.close()`.

```

plt.figure(dpi=150)
E_Ry= -13.312622836061621 , Standardfehler= 0.2309511947591827
E_3p= -3.023694832517033 , Standardfehler= 0.003775784601552789
D_d= 0.03088361539955528 , Standardfehler= 0.02276220634648546
chi2= 2.5349938374137855
chi2_red= 0.4224989729022976
Wahrscheinlichkeit: 86.0 %

```

1. Nebenserie des Na-Atoms



Fitten der zweiten Nebenserie

```
In [30]: def fit_func2(m,E_Ry,E_3p,korrs):
          return 1.2398E3/(E_Ry/(m-korrs)**2-E_3p)
          para = [-13.6,-3,-0.02]
          popt, pcov = curve_fit(fit_func2,Nebenserie2m,Nebenserie2_wellen,sigma=Nebenserie2_wellenfehler,p0=para)
          x=np.linspace(4.8,13.2, 100)

          plt.figure(dpi=150)
```

```

plt.errorbar(Nebenserie2m,Nebenserie2_wellen,yerr=Nebenserie2_wellenfehler,fmt=".",capsize=3,elinewidth=1,label='Messwerte')
plt.plot(x,fit_func1(x,*popt))
plt.xlabel('Quantenzahl')
plt.ylabel('Wellenlänge [nm]')
plt.title('2. Nebenserie des Na-Atoms')
plt.legend()
plt.grid()
plt.savefig(Ausgabe_path/"Nr.3_Nebenserie2.png", format="png")

print("E_Ry=",popt[0], ", Standardfehler=", np.sqrt(pcov[0][0]))
print("E_3p=",popt[1], ", Standardfehler=", np.sqrt(pcov[1][1]))
print("D_s=",popt[2], ", Standardfehler=", np.sqrt(pcov[2][2]))

chi2_=np.sum((fit_func1(Nebenserie2m,*popt)- Nebenserie2_wellen)**2/ Nebenserie2_wellenfehler**2)
dof=len(Nebenserie2m)-3 #dof:degrees of freedom, Freiheitsgrad
chi2_red=chi2_/dof
print("chi2=", chi2_)
print("chi2_red=",chi2_red)

prob=round(1-chi2_.cdf(chi2_,dof),2)*100
print("Wahrscheinlichkeit:", prob,"%")

```

```

E_Ry= -15.424418960685733 , Standardfehler= 2.491190299549173
E_3p= -3.0596767847265967 , Standardfehler= 0.03237167290981489
D_s= 1.1628769594041732 , Standardfehler= 0.25291932741264594
chi2= 6.302600904126201
chi2_red= 3.1513004520631007
Wahrscheinlichkeit: 4.0 %

```

2. Nebenserie des Na-Atoms

